

NoteFlow — ход разработки

1. Проектирование архитектуры

Первым делом определился с составом системы. Решил разделить на три микросервиса по зонам ответственности: пользователи, заметки, файлы. Каждый сервис — отдельное FastAPI-приложение со своей БД.

Выбор стека для каждого сервиса:

- user-service: PostgreSQL, потому что данные пользователей строго структурированы и нужна консистентность (уникальность email, внешние ключи в будущем).
- note-service: MongoDB, потому что заметки имеют переменную структуру — список тегов произвольной длины, список вложений, и гибкие поля проще добавлять без миграций схемы.
- file-service: PostgreSQL для метаданных + MinIO для самих байтов. Метаданные структурированы, а хранить файлы в БД — плохая практика.

Для связи между сервисами выбрал Kafka. Альтернативы рассматривались: RabbitMQ проще, но нет персистентного лога; NATS ещё проще и быстрее, но JetStream добавляет сложности. Kafka избыточна для этого масштаба, но даёт возможность дочитать события после перезапуска сервиса и читать один топик несколькими независимыми consumer group.

2. Создание скелетов сервисов

Начал с создания структуры директорий для каждого сервиса:

```
{service}/  
  app/  
    config.py  
    main.py  
    dependencies.py  
    models/  
    schemas/  
    routers/  
    services/  
    repositories/  
    kafka/
```

```
pyproject.toml
Dockerfile
```

Каждый сервис использует `uv` для управления зависимостями. `pyproject.toml` описывает зависимости, `Dockerfile` строит образ на `python:3.12-slim`.

Конфигурация через `pydantic-settings`: все переменные окружения читаются из `.env` или переменных среды, с дефолтными значениями для локальной разработки.

3. Реализация user-service

Начал с модели пользователя. SQLAlchemy с `asyncpg`. Модель `User` содержит `id` (UUID), `email`, `pass_hash`, `display_name`, `avatar_url`, `created_at`, `deleted_at`. Soft delete через поле `deleted_at`.

Для хэширования паролей - `bcrypt`: `hashpw/checkpw`.

JWT: access token (HS256, TTL 15 мин) содержит `sub` (`user_id`) и `jti` (уникальный ID токена). `jti` нужен для `blacklist` при логауте.

Refresh токены изначально хотел хранить в PostgreSQL отдельной таблицей. В процессе приняли решение перенести их в Redis — это проще, быстрее, и TTL управляется автоматически без джобов очистки. Схема ключей:

- `refresh:{hash}` — данные токена с TTL 30 дней
- `sessions:{user_id}` — Set из хэшей всех активных сессий пользователя

Это позволяет отозвать все сессии одной операцией (`SMEMBERS` + `DEL` каждого ключа).

`get_redis()` реализован как `lazy singleton` в `dependencies.py` — создаётся при первом вызове, переиспользуется далее.

Роутеры: `/auth` (`register`, `login`, `refresh`, `logout`) и `/users` (`me` GET/PATCH/DELETE). Refresh token передаётся в `httpOnly cookie` с `path=/auth/refresh`, чтобы браузер не отправлял его на другие эндпоинты.

При логауте: `jti` записывается в `jwt:blacklist:{jti}` с TTL = оставшееся время жизни токена, refresh token удаляется из Redis, cookie очищается.

4. Docker Compose и инфраструктура

Написал `docker-compose.yml` с полным стеком:

- postgres-users (5432), postgres-files (5433) — два отдельных инстанса PostgreSQL
- mongo (27017)
- redis (6379)
- kafka (9092)
- minio (9000 API, 9001 console)

Для Kafka изначально пробовал bitnami/kafka, но образ не нашёлся. Перешел на apache/kafka:3.7.0 с KRaft-режимом (без Zookeeper).

Healthcheck для каждого сервиса инфраструктуры.

Для hot-reload без пересборки контейнера: монтируем ./service/app:/app/app как volume и запускаем uvicorn с флагом --reload. Пересборка нужна только при изменении pyproject.toml.

5. Реализация note-service

Выбрали Beanie как ODM для MongoDB. Возникла проблема: Beanie 2.x больше не использует Motor под капотом, перешёл на нативный async-клиент PyMongo.

Для поля user_id использовали Annotated[str, Indexed()] — новый синтаксис Beanie 2.x вместо устаревшего Indexed(str).

Функциональность: CRUD заметок, soft delete + корзина + восстановление, история версий (последние 20), теги с подсчётом количества заметок, полнотекстовый поиск через MongoDB text index.

Kafka consumer в note-service слушает:

- user.registered — создаёт приветственную заметку
- user.deleted — soft delete всех заметок пользователя
- file.uploaded — прикрепляет файл к заметке (находит заметку по note_ref из события)

6. Реализация file-service

Для работы с MinIO используется aiobotocore — async-обёртка над boto3 с S3-совместимым API. Клиент создаётся через context manager get_s3_client().

При загрузке файла: стримим multipart в MinIO, сохраняем метаданные (owner_id, original_name, mime_type, size_bytes, bucket, object_key) в PostgreSQL.

Object key строится как `{owner_id}/{file_id}_{original_name}` — это позволяет видеть структуру в MinIO по пользователям.

Presigned URL генерируется через `generate_presigned_url` с `ExpiresIn=3600`. URL содержит `AWSSecretAccessKey` (публичная часть `credentials`) и `Signature` (HMAC-подпись, вычисленная секретным ключом).

Kafka consumer слушает `user.deleted` — удаляет файлы из MinIO и помечает записи в БД как `deleted`.

7. Swagger UI и авторизация

FastAPI генерирует документацию автоматически по роутерам и схемам. Для отображения кнопки `Authorize` в Swagger и поддержки JWT добавили `HTTPBearer` в каждый сервис.

8. Nginx и rate limiting

Добавили Nginx как единую точку входа. Конфиг: `upstream`-блоки для каждого сервиса, `location`-блоки с маршрутизацией по префиксу пути. Nginx выставляет `X-Real-IP` и `X-Forwarded-For`.

Rate limiting реализован как `BaseHTTPMiddleware` в каждом FastAPI-приложении. Использует Redis.

Правила для `user-service`:

- `POST /auth/login`: 5 запросов за 15 минут (защита от брутфорса)
- `POST /auth/register`: 10 запросов за час
- остальное: 100 запросов в минуту

Для `note-service` и `file-service`: 100 запросов в минуту.

При превышении — 429 с заголовками `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`. Флаг `rate_limit_enabled` в конфиге позволяет отключить в тестовом окружении.